Foto - Personalized Photography Coaching Platform

Complete System Documentation

Version: 2024112504
Last Updated: November 25, 2025
Status: Production (Live at https://myfoto.topues.com)

📋 Table of Contents

1. Overview
2. Technology Stack
3. Architecture
4. Features
5. Database Schema
6. API Endpoints
7. External Integrations
8. Frontend Components
9. AI/LLM Integration
10. Email System
11. Deployment
12. File Structure

🎯 Overview

Foto is a personalized photography coaching platform designed for Seun Salami, a Sydney/Melbourne-based photographer. The platform combines AI-powered coaching with practical photography challenges tailored to the user's gear, skill level, schedule, and local weather conditions.

Core Value Proposition

- **Personalized Daily Challenges**: AI-generated photography tasks based on skill level, interests, and available gear
- **Location-Based Itineraries**: Custom photography routes for any location with optimal timing
- **AI Chatbot Mentor**: Embedded conversational AI for real-time photography coaching

- **Progress Tracking**: Streak tracking, completion stats, and skill growth metrics
- **Weather-Aware**: Integrates live weather data to suggest optimal shooting conditions
- **One-Click Email Delivery**: Send challenges directly to email for offline reference

---

🛠 Technology Stack

Frontend

Technology	Version	Purpose
Next.js	14.2.28	React framework with App Router
React	18.2.0	UI library
TypeScript	5.2.2	Type safety
Tailwind CSS	3.3.3	Utility-first styling
Framer Motion	10.18.0	Animations
Radix UI	Multiple	Accessible UI primitives
Lucide React	0.446.0	Icon library
Recharts	2.15.3	Data visualization
React Hook Form	7.53.0	Form handling
Zod	3.23.8	Schema validation

Backend

Technology	Version	Purpose
Next.js API Routes	14.2.28	Serverless API endpoints
Prisma ORM	6.7.0	Database access layer
PostgreSQL	Latest	Relational database
NextAuth.js	4.24.11	Authentication
bcryptjs	2.4.3	Password hashing
jsonwebtoken	9.0.2	JWT tokens

External Services

Service	Purpose
Abacus.AI	LLM API for AI features
SendGrid	Transactional email
OpenWeatherMap	Weather data API

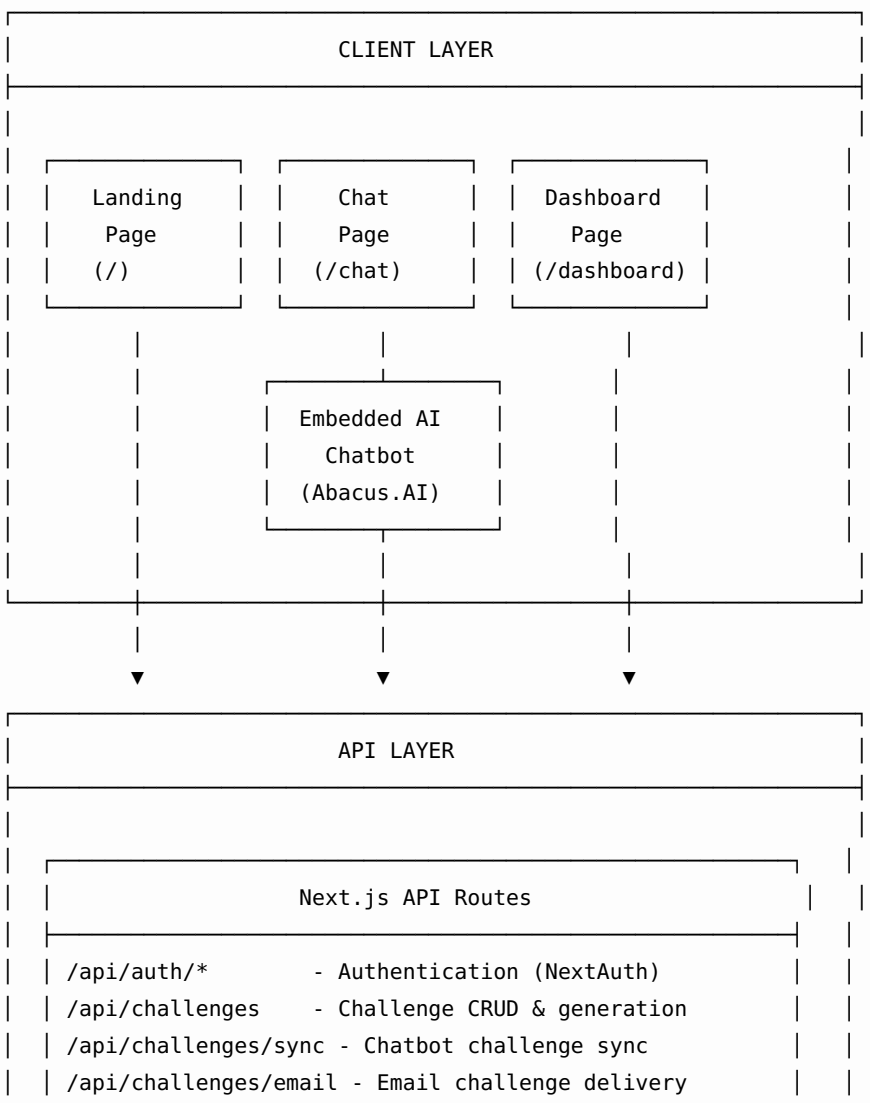
Abacus.AI ChatLLM	Embedded chatbot

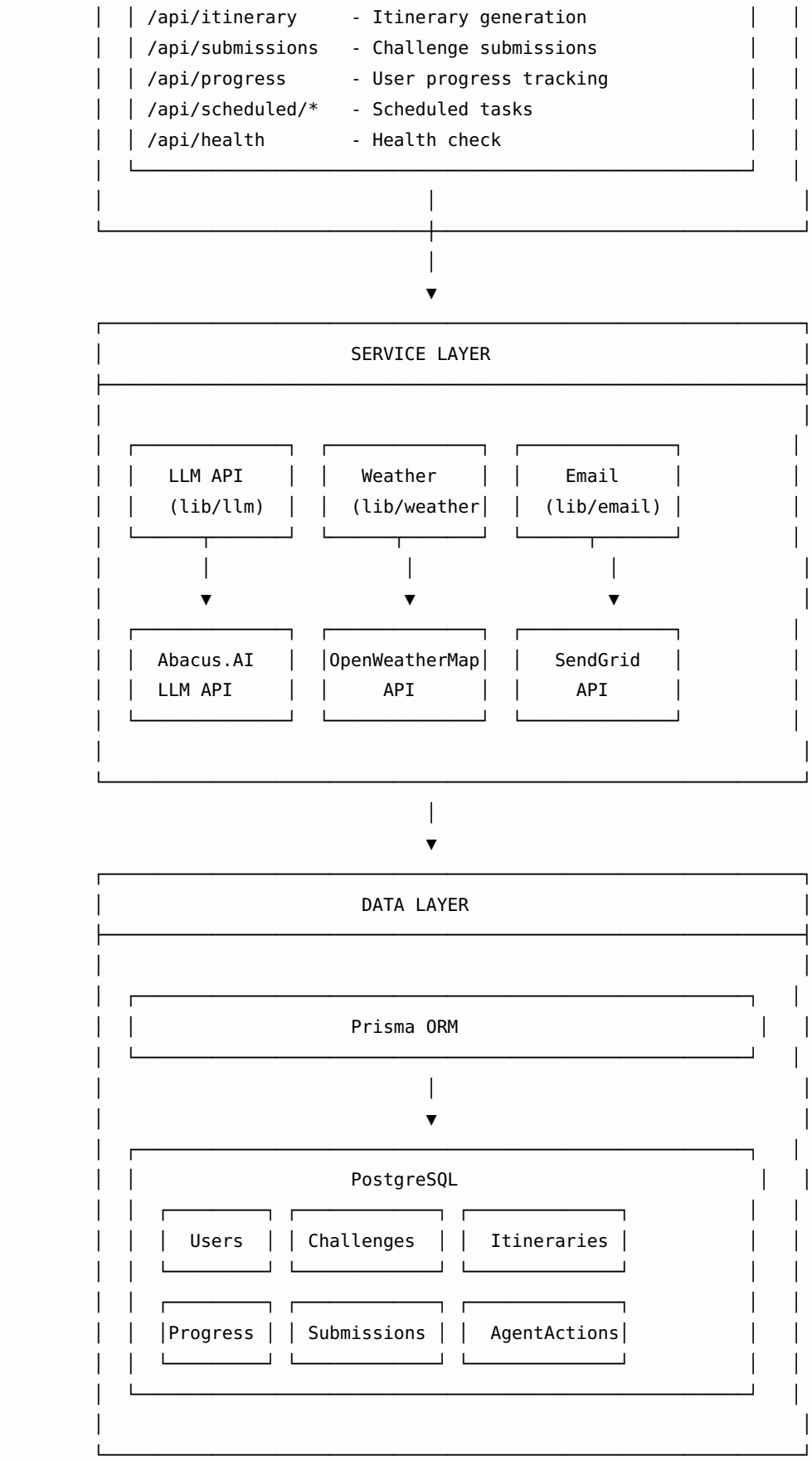
DevOps & Deployment

Technology	Purpose
Docker	Containerization
Docker Compose	Multi-container orchestration
Abacus.AI DeepAgent	Production hosting
yarn	Package manager

🏗 Architecture

High-Level Architecture




/api/itinerary - Itinerary generation
/api/submissions - Challenge submissions
/api/progress - User progress tracking
/api/scheduled/* - Scheduled tasks
/api/health - Health check
SERVICE LAYER
LLM API (lib/llm)
Weather (lib/weather)
Email (lib/email)
Abacus.AI LLM API
OpenWeatherMap API
SendGrid API
DATA LAYER
Prisma ORM
PostgreSQL
Users
Challenges
Itineraries
Progress
Submissions
AgentActions

Data Flow

Challenge Generation Flow

```
User Request → API Route → LLM Service → Weather API → Generate Challenge → Database → Response
```

Challenge Sync Flow (Chatbot → Dashboard)

```
Chatbot generates challenge → JavaScript sync script detects → POST /api/challenges/sync → Database → Dashboard polls & displays
```

Email Challenge Flow

```
User clicks email button → Extract challenge data → POST /api/challenges/email → SendGrid API → Email delivered
```

✨ Features

1. AI Photography Chatbot

Location: `/chat` page
Implementation: Embedded Abacus.AI ChatLLM chatbot

Capabilities: - Generate personalized photography challenges - Create location-based itineraries - Provide coaching advice and tips - Answer photography-related questions - Suggest gear recommendations

Technical Details: - Chatbot embedded via iframe - Challenge sync script (`/public/email-challenge.js`) monitors chatbot responses - Automatic challenge extraction and dashboard sync - One-click email delivery button

2. Daily Photography Challenges

Challenge Types:

Type	Duration	Complexity	Description
`WEEKEND_DEEP_DIVE`	2-4 hours	High	Substantial creative project for weekends
`LUNCH_MICRO`	15-25 mins	Micro	Quick exercise during lunch break
`AFTER_WORK`	30-60 mins	Short	Evening photography

			session
`WORK_FROM_HOME`	45-90 mins	Medium	Flexible home/neighborhood project

Challenge Categories: - PORTRAIT - LANDSCAPE - STREET - MARKET - ARCHITECTURE - NATURE - ABSTRACT

Difficulty Levels: - BEGINNER - INTERMEDIATE - ADVANCED - MINI

3. Location-Based Itineraries

Features: - AI-generated photography routes - 3-5 specific photo spots per itinerary - Optimal timing recommendations (golden hour, etc.) - Weather-aware suggestions - Gear recommendations per location

Data Structure:

```
{
  "title": "Melbourne Street Photography Tour",
  "description": "Explore Melbourne's urban landscape",
  "spots": [
    {
      "name": "Hosier Lane",
      "location": "Hosier Lane, Melbourne CBD",
      "bestTime": "Golden hour 5-6 PM",
      "photoOpportunities": ["street art", "portraits", "urban
        texture"],
      "tips": "Use wide angle for full murals"
    }
  ],
  "suggestedTimes": ["Start at 4 PM", "End by sunset"],
  "gearRecommendations": ["35mm lens", "polarizer"]
}
```

4. Progress Tracking Dashboard

Location: `/dashboard` page

Metrics Tracked: - Total challenges completed - Current streak - Longest streak - Challenges by type (weekend, lunch, after-work, WFH) - Favorite subjects - Improvement areas - Last challenge date

Dashboard Tabs: 1. **Current Challenge** - Active challenge with completion form 2. **History** - Past challenges and submissions 3. **Itinerary** - Generated photography routes

5. Challenge Submission System

Features: - Photo upload support - Notes/reflection field - Rating system - AI-generated feedback - Automatic streak calculation

6. One-Click Email Challenge

Version: 2024112504

How It Works: 1. Script monitors chat page for challenge content 2. Detects challenge keywords (challenge, difficulty, photography, etc.) 3. Displays “Email This Challenge” button 4. On click, extracts challenge data 5. Sends via `/api/challenges/email` endpoint 6. Email delivered via SendGrid

Target Email: seun_salami@outlook.com

7. Weather Integration

Features: - Real-time weather data fetching - Photography condition analysis (score 0-100) - Suggested photography styles based on weather - Golden hour calculations - Weather alerts (rain, extreme temps, fog opportunities)

Weather Scoring Factors: - Temperature comfort - Cloud coverage (overcast = good for portraits) - Special conditions (fog, rain = photo opportunities) - Visibility - Wind speed - Golden/blue hour timing

8. User Preferences System

Configurable Options: - Work schedule (days, times) - Photography availability windows - Skill level - Photography interests - Avoided styles - Challenge complexity preferences - Notification settings - Location preferences

9. Gear Inventory Management

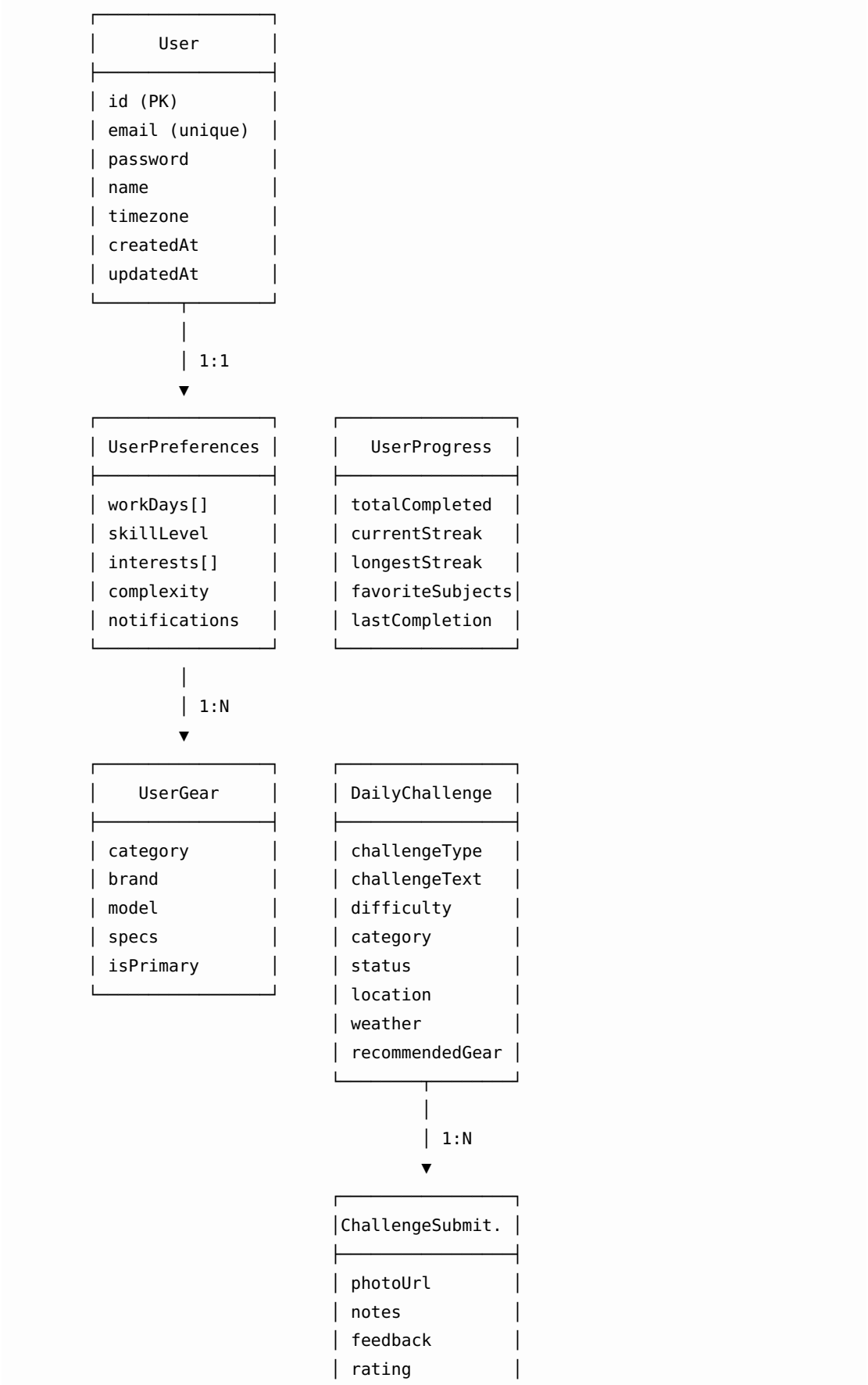
Stored Information: - Category (camera, lens, accessory) - Brand and model - Specifications - Primary gear flag

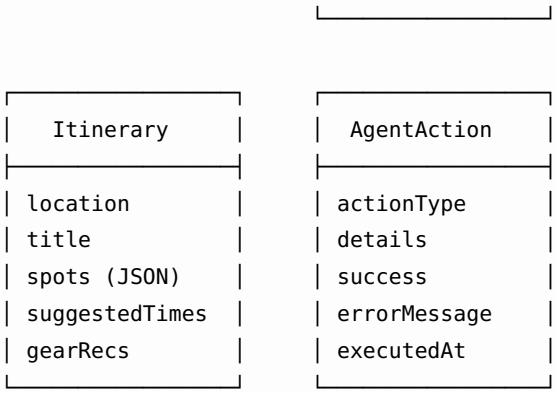
Usage: - Challenges recommend specific gear from user’s inventory - Itineraries suggest appropriate equipment - AI considers available gear for challenge feasibility

---

🗄 Database Schema

Entity Relationship Diagram


User
id (PK)
email (unique)
password
name
timezone
createdAt
updatedAt
1:1
UserPreferences
workDays[]
skillLevel
interests[]
complexity
notifications
UserProgress
totalCompleted
currentStreak
longestStreak
favoriteSubjects
lastCompletion
1:N
UserGear
category
brand
model
specs
isPrimary
DailyChallenge
challengeType
challengeText
difficulty
category
status
location
weather
recommendedGear
1:N
ChallengeSubmit.
photoUrl
notes
feedback
rating



Models Summary

Model	Purpose	Key Fields
User	User accounts	email, password, name, timezone
UserPreferences	User settings	skillLevel, interests, schedule, notifications
UserGear	Equipment inventory	category, brand, model, isPrimary
DailyChallenge	Photography challenges	challengeText, difficulty, status, location
ChallengeSubmission	Completed challenges	photoUrl, notes, feedback, rating
Itinerary	Photography routes	location, spots, suggestedTimes
UserProgress	Stats & streaks	totalCompleted, currentStreak, longestStreak
AgentAction	Audit log	actionType, details, success

🔌 API Endpoints

Authentication

Endpoint	Method	Description
`/api/auth/[...nextauth]`	ALL	NextAuth.js handler
`/api/signup`	POST	User registration

Challenges

Endpoint	Method	Description
`/api/challenges`	GET	Get user's challenges
`/api/challenges`	POST	Generate new challenge
`/api/challenges/[id]`	GET	Get specific challenge
`/api/challenges/[id]`	PATCH	Update challenge status
`/api/challenges/sync`	POST	Sync challenge from chatbot
`/api/challenges/email`	POST	Email challenge to user

Itineraries

Endpoint	Method	Description
`/api/itinerary`	GET	Get user's itineraries
`/api/itinerary`	POST	Generate new itinerary

Submissions

Endpoint	Method	Description
`/api/submissions`	GET	Get user's submissions
`/api/submissions`	POST	Submit completed challenge

Progress

Endpoint	Method	Description
`/api/progress`	GET	Get user progress stats

Scheduled Tasks

Endpoint	Method	Description
`/api/scheduled/daily-challenge`	GET	Generate daily challenge
`/api/scheduled/streak-manager`	GET	Update streak calculations
`/api/scheduled/weekly-report`	GET	Send weekly progress report
`/api/scheduled/weather-monitor`	GET	Check weather alerts

Utility

Endpoint	Method	Description
`/api/health`	GET	Health check
`/api/email-test`	GET	Test email configuration

🔗 External Integrations

1. Abacus.AI LLM API

Purpose: AI-powered challenge generation, feedback, and itinerary creation

Endpoint: `https://apps.abacus.ai/v1/chat/completions`

Model: `gpt-4.1-mini`

Functions: - `callLLM()` - Base LLM call - `generateChallenge()` - Create photography challenges - `generateFeedback()` - AI feedback on submissions - `generateItinerary()` - Location-based routes

Configuration: - Temperature: 0.7-0.8 - Max tokens: 300-1500 depending on task - JSON mode supported

2. Abacus.AI ChatLLM (Embedded Chatbot)

Purpose: Conversational AI photography coach

Integration: iframe embed on `/chat` page

Features: - Real-time conversation - Challenge generation - Photography advice - Gear recommendations

Sync Mechanism: - JavaScript detection script - Automatic challenge extraction - Dashboard sync via API

3. SendGrid Email API

Purpose: Transactional email delivery

Features: - Challenge notifications - Weekly progress reports - Weather alerts - One-click challenge emails

Email Templates: - HTML with inline CSS - Plain text fallback - Branded design with gradients - Responsive layout

Sender: seun@topues.com (verified)

4. OpenWeatherMap API

Purpose: Weather data for challenge customization

Features: - Current conditions - Temperature, humidity, wind - Cloud coverage - Sunrise/sunset times - Visibility

Analysis Functions: - `getWeather()` - Fetch current weather - `analyzeWeatherForPhotography()` - Photography score - `getGoldenHourTimes()` - Optimal shooting times

---

🎨 Frontend Components

Pages

Page	Path	Description
Home	`/`	Landing page with hero, features, CTA
About	`/about`	About Seun and the platform
Chat	`/chat`	AI chatbot interface
Dashboard	`/dashboard`	Challenge tracking & management

Section Components (`/components/sections/`)

Component	Description
`hero.tsx`	Main landing hero section
`features.tsx`	Platform features showcase
`how-it-works.tsx`	Step-by-step guide
`call-to-action.tsx`	CTA buttons
`about-hero.tsx`	About page hero
`seun-profile.tsx`	Coach profile
`coaching-approach.tsx`	Methodology explanation
`photography-styles.tsx`	Style showcase
`gear-showcase.tsx`	Equipment display
`chat-interface.tsx`	Chatbot container

UI Components (`/components/ui/`)

61 total components including:

Category	Components
Layout	Card, Dialog, Sheet, Drawer, Separator
Forms	Input, Textarea, Select, Checkbox, Radio Group, Switch

Navigation	Tabs, Accordion, Navigation Menu, Breadcrumb
Feedback	Alert, Badge, Progress, Skeleton, Toast
Overlay	Popover, Tooltip, Hover Card, Context Menu
Data Display	Table, Avatar, Calendar, Date Range Picker
Interactive	Button, Toggle, Slider, Carousel

Client-Side Scripts (`/public/`)

Script	Purpose
`email-challenge.js`	Email button & challenge detection
`challenge-sync.js`	Chatbot → Dashboard sync
`challenge-sync-v2.js`	Enhanced sync with JSON hiding

🤖 AI/LLM Integration

Challenge Generation Prompt Structure

```
System: You are Foto AI, an expert photography coach...

User Profile:
- Skill Level: {intermediate}
- Interests: {street, portrait, architecture}
- Available Gear: {Sony A7III, 35mm f/1.8, ...}

Challenge Context:
- Type: {LUNCH_MICRO}
- Complexity: {micro - Quick 15-20 minute exercise}
- Duration: {15-25 mins}
- Location: {Melbourne CBD}
- Weather: {Clear, 72°F}

Recent challenges to avoid: {...}

Generate a specific, actionable challenge...
```

Response Format

```json
{
  "challengeText": "Clear, specific challenge description",
  "difficulty": "micro|short|medium|deep",
  "category": "street|portrait|architecture|...",
  "duration": "15-25 mins",
```

```
  "tips": "Practical execution tips",
  "recommendedGear": ["Sony A7III", "35mm f/1.8"]
}
```

Itinerary Generation

- Location-specific spots (verified locations)
- Optimal timing for each spot
- Photo opportunities list
- Gear recommendations per location
- Weather-aware suggestions

Feedback Generation

- Personalized to skill level
- Acknowledges effort
- Highlights positives
- 1-2 improvement suggestions
- Encouraging tone

📧 Email System

Email Types

Type	Trigger	Content
Daily Challenge	New challenge generated	Full challenge details, tips, gear
Weekly Report	Scheduled	Stats summary, streak info
Weather Alert	Special conditions	Photography opportunity notification
Challenge Email	User clicks button	Challenge from chatbot

Email Template Design

- **Header:** Gradient background (purple/blue)
- **Content:** Clean white cards
- **Badges:** Category, difficulty, duration
- **Tips Section:** Yellow highlight box
- **CTA Button:** Link to dashboard
- **Footer:** Unsubscribe info

One-Click Email Flow

```
1. User sees challenge in chatbot
2. email-challenge.js detects challenge content
3. "Email This Challenge" button appears
4. User clicks button
5. Script extracts: title, category, difficulty, duration, tips,
gear
6. POST to /api/challenges/email
7. API formats HTML/text email
8. SendGrid delivers to seun_salami@outlook.com
9. Button shows success state
```

🚀 Deployment

Production Environment

- **Platform:** Abacus.AI DeepAgent
- **URL:** https://myfoto.topues.com
- **Build:** Next.js standalone output
- **Database:** Hosted PostgreSQL

Build Process

```
# Build command (automated)
yarn run build

# Output: .build/standalone
# Includes: public/, uploads/
```

Environment Variables

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string
`NEXTAUTH_SECRET`	Auth encryption key
`NEXTAUTH_URL`	Base URL for auth
`ABACUSAI_API_KEY`	LLM API access
`SENDGRID_API_KEY`	Email API access
`OPENWEATHERMAP_API_KEY`	Weather API access
`FROM_EMAIL`	Email sender address
`USER_EMAIL`	Default recipient

Docker Support

```
# docker-compose.yml services:
```

- app (Next.js application)
- postgres (PostgreSQL database)

Scripts Available

Script	Purpose
`deploy.sh`	Full deployment
`backup.sh`	Database backup
`restore.sh`	Restore from backup
`monitor.sh`	Health monitoring

📁 File Structure

```
/home/ubuntu/foto_app/
├── nextjs_space/                # Main application
│   ├── app/                     # Next.js App Router
│   │   ├── page.tsx             # Landing page
│   │   ├── about/page.tsx       # About page
│   │   ├── chat/page.tsx        # Chat page
│   │   ├── dashboard/page.tsx   # Dashboard page
│   │   ├── layout.tsx           # Root layout
│   │   ├── globals.css          # Global styles
│   │   └── api/                 # API routes
│   │       ├── auth/            # Authentication
│   │       ├── challenges/      # Challenge endpoints
│   │       ├── itinerary/       # Itinerary endpoints
│   │       ├── submissions/     # Submission endpoints
│   │       ├── progress/        # Progress endpoints
│   │       └── scheduled/       # Scheduled tasks
│   │
│   ├── components/              # React components
│   │   ├── sections/            # Page sections (10)
│   │   ├── ui/                  # UI primitives (51)
│   │   ├── navbar.tsx           # Navigation
│   │   ├── footer.tsx           # Footer
│   │   └── theme-provider.tsx   # Theme context
│   │
│   ├── lib/                     # Utilities
│   │   ├── auth.ts              # NextAuth config
│   │   ├── db.ts                # Prisma client
│   │   ├── llm.ts               # LLM API functions
│   │   ├── email.ts             # SendGrid functions
│   │   ├── weather.ts           # Weather API functions
│   │   ├── types.ts             # TypeScript types
│   │   └── utils.ts             # Helper functions
```

```
│   │
│   ├── prisma/
│   │   └── schema.prisma         # Database schema
│   │
│   ├── public/
│   │   ├── email-challenge.js    # Email button script
│   │   ├── challenge-sync.js     # Sync script
│   │   └── uploads/              # User uploads
│   │
│   ├── scripts/
│   │   └── seed.ts               # Database seeding
│   │
│   ├── hooks/
│   │   └── use-toast.ts          # Toast hook
│   │
│   ├── package.json              # Dependencies
│   ├── next.config.js            # Next.js config
│   ├── tailwind.config.ts        # Tailwind config
│   ├── tsconfig.json             # TypeScript config
│   └── .env                      # Environment vars
│
├── *.md                          # Documentation files
├── deploy.sh                     # Deployment script
├── backup.sh                     # Backup script
├── restore.sh                    # Restore script
└── monitor.sh                    # Monitoring script
```

📊 Key Metrics & Limits

Metric	Value
Total React Components	61+
API Endpoints	18
Database Models	8
Challenge Types	4
Challenge Categories	7
Email Types	4
LLM Calls	Challenge gen, feedback, itinerary

🔐 Security Features

- Password hashing with bcryptjs
- JWT token authentication

- NextAuth.js session management
- Environment variable protection
- API route validation
- Prisma parameterized queries

📝 Recent Updates (v2024112504)

1. **Email URL Fix** - Changed hardcoded URL to relative path
2. **Flexible Challenge Detection** - Button appears for all challenges
3. **SendGrid Integration** - Verified sender configured
4. **Challenge Sync** - Automatic chatbot → dashboard sync
5. **Skip Challenge** - Users can skip current challenges
6. **Dashboard Polling** - Auto-refresh for new challenges

🗃 Documentation Files

File	Purpose
`README.md`	Project overview
`SYSTEM_DOCUMENTATION.md`	This file
`SELF_HOSTING_GUIDE.md`	Self-deployment guide
`GITHUB_SETUP_INSTRUCTIONS.md`	Repository setup
`CHALLENGE_SYNC_INTEGRATION.md`	Sync system docs
`EMAIL_*.md`	Email feature docs
`CHATBOT_*.md`	Chatbot setup docs

🎯 Summary

Foto is a full-stack personalized photography coaching platform featuring:

✅ AI-powered daily challenges
✅ Embedded conversational chatbot
✅ Location-based itineraries
✅ Progress tracking with streaks
✅ Weather-aware recommendations
✅ Gear-specific suggestions
✅ One-click email delivery
✅ Challenge submission with feedback
✅ Beautiful, responsive UI
✅ Production-ready deployment

Built with: Next.js 14, TypeScript, Prisma, PostgreSQL, Tailwind CSS, Abacus.AI, SendGrid, OpenWeatherMap

Live at: https://myfoto.topues.com

Documentation generated November 25, 2025